Inverted Indexing, Sparse Retrieval, and the Vocabulary Mismatch Problem

ROLL1 ROLL2

5 September 2026

Datasets

BEIR test splits loaded with `ir_datasets` (`beir/<name>/test`). All runs on one laptop under WSL2 (Intel Core Ultra 7 155U, 7.6 GB RAM for WSL, no GPU; Pyserini 2.3.0, Lucene 10, Java 21, PyTorch CPU).

- **SciFact**: 5 183 documents, 300 test queries, 339 qrels. Standard split, no deviation.
- **FEVER**: 5 416 568 documents, 6 666 test queries, 7 937 qrels. Parts 1–3 on all queries; Parts 4–5 on a fixed random sample of 500 test queries (seed 0), because feedback and LLM/SPLADE encoding run per query on CPU.
- **HotpotQA**: 5 233 329 documents, 7 405 test queries, 14 810 qrels (two gold passages per question). Same 500-query sample for Parts 4–5.

Metrics: nDCG@10, Recall@100, MRR@10, MAP with `ir_measures` on top-100 runs. Every table states the number of queries used, and each baseline is recomputed on the same queries as the method it is compared with.

1 Part 1: Index

One Lucene index per corpus with `python -m pyserini.index.lucene -collection JsonCollection -storeDocvectors` (`contents` = title + text; default English analyzer; document vectors stored for Part 4). Build time is the wall-clock time of the indexing process alone (corpus dump excluded), measured with no other job running. `LuceneIndexReader` gives the statistics Part 4a needs: term-frequency vector (`get_document_vector`), document length (its sum) and corpus document frequency (`get_term_counts`); Table 2 shows them for one gold document.

Dataset	Documents	Test queries	Unique terms	Build time	Index size
SciFact	5,183	300	28,865	0.1 min	4.7 MB
FEVER	5,416,568	6666	3,293,639	18.5 min	2.30 GB
HotpotQA	5,233,329	–	2,644,892	11.9 min	1.22 GB

Table 1: Part 1: Lucene index per dataset (Pyserini `JsonCollection`, `-storeDocvectors`, 4 indexing threads).

2 Part 2: Sparse retrieval baselines

`LuceneSearcher` over each Part 1 index: (a) Pyserini default BM25 (k_1=0.9, b=0.4); (b) BM25 with k_1, b chosen per dataset by an exhaustive 5×5 grid over all test queries, selecting by nDCG@10 (Table 4); (c) classic TF-IDF by setting Lucene's `ClassicSimilarity` on the underlying `IndexSearcher`.

Dataset	Gold doc id	Length	Distinct terms	Top-tf terms (tf, corpus df)
SciFact	31715818	64	46	cell (tf=5, df=2,530), stem (tf=5, df=4 otechnolog (tf=3, df=4), track (tf=3, df (tf=3, df=2,238)
FEVER	Ukrainian_Soviet_Socialist_Republic	210	137	soviet (tf=12, df=40,403), republ df=88,471), ukrainian (tf=7, df=15,079 (tf=7, df=17,073), union (tf=6, df=112
HotpotQA	2816539	39	36	scott (tf=2, df=15,764), derrickson df=10), he (tf=2, df=673,663), insta df=14,083), lo (tf=1, df=37,275)

Table 2: Part 1: IndexReader statistics for the first gold document of each dataset (analysed, stemmed terms).

Dataset	Model	nDCG@10	R@100	MRR@10	MAP
SciFact	BM25 default (k_1=0.9, b=0.4)	0.6789	0.9253	0.6457	0.6398
	BM25 tuned (k_1=2.0, b=0.75)	0.6892	0.9282	0.6525	0.6452
	TF-IDF (Lucene `ClassicSimilarity`)	0.6744	0.9032	0.6390	0.6277
FEVER	BM25 default (k_1=0.9, b=0.4)	0.6513	0.9185	0.6230	0.5988
	BM25 tuned (k_1=0.9, b=0.3)	0.6716	0.9226	0.6459	0.6194
	TF-IDF (Lucene `ClassicSimilarity`)	0.2075	0.6637	0.1727	0.1779

Table 3: Part 2: sparse baselines on the full BEIR test query sets.

PART2DISCUSSION

3 Part 3: Vocabulary mismatch

For every (query, gold document) pair, BM25 (tuned) *fails* if the gold document is not in the top-10. Lexical overlap = Jaccard coefficient between the analysed query tokens and the gold document's indexed term vector (same analyzer as the index). Failures are bucketed as: *no lexical overlap* (no shared analysed term); *abbreviation vs. expansion* (an all-caps token on one side whose initials appear as consecutive words on the other); *partial overlap / paraphrase* (fewer than half of the query terms in the gold document); *high overlap but outranked* (at least half of the query terms present, yet other documents score higher).

Figure 1: Part 3: distribution of Jaccard overlap for pairs where BM25 finds the gold document in the top-10 vs. misses it.

Verdict. PART3VERDICT

4 Part 4: Pseudo-relevance feedback

4.1 4a. Rocchio and RM3 (corpus-retrieved feedback)

Own implementation (`rocchio.py`):

$$w_t = \alpha\, f(q)[t] + \frac{\beta}{N} \sum_{d \in \text{top-}N} \tilde{f}(d)[t], \qquad \alpha = 1,\ \beta = 0.75,$$

Dataset	Best nDCG@10 (k_1,b)	Worst nDCG@10 (k_1,b)	Default	BM25 latency (ms/query)
SciFact	0.6892 (2.0,0.75)	0.6773 (0.6,0.4)	0.6789	5.44
FEVER	0.6716 (0.9,0.3)	0.3257 (2.0,1.0)	0.6513	6.14

Table 4: Part 2: grid search over $k_1 \in \{0.6, 0.9, 1.2, 1.5, 2.0\}$, $b \in \{0.3, 0.4, 0.5, 0.75, 1.0\}$ on all test queries (nDCG@10); latency = batch BM25 search with 8 threads.

Dataset	no overlap	abbrev. vs exp.	partial overlap	high overlap, outranked	total

Table 5: Part 3: failure cases by (heuristic) category.

with $f(q)$ and $\tilde{f}(d)$ length-normalised term frequencies (tf/$|d|$). Feedback vectors are the Part 1 term vectors of the top-N tuned-BM25 documents (`IndexReader.get_document_vector`); terms in more than 10% of the documents (`get_term_counts`) are never added; the k heaviest remaining centroid terms are kept. The re-weighted query is built with Pyserini's query builder as a Boolean OR of `BoostQuery(TermQuery(contents:t), w_t)` and re-searched. RM3: `set_rm3(fb_terms=k, fb_docs=N, original_query_weight=0.5)`.

Query drift. Queries with the largest nDCG@10 drop after Rocchio, with the added terms:
PART4ADRIFT

4.2 4b. HyDE (LLM-generated feedback)

Hypothetical documents are sampled locally on CPU with Qwen2.5-0.5B-Instruct (Transformers; T=0.7, top-p 0.9; two 96-token documents per query for SciFact, one 64-token document per query for the FEVER/HotpotQA samples), prompted to write a passage that supports/refutes the claim or answers the question. The unchanged `rocchio.py` then receives the analysed hypothetical documents as its feedback vectors. Variants on the same queries as 4a: naive concatenation (query + hypothetical documents as one BM25 query), HyDE + Rocchio ($k \in \{10, 20\}$), and 4a's corpus PRF.

Feedback source vs. combination method. PART4BVERDICT

5 Part 5: SPLADE

Checkpoint `naver/splade-cocondenser-ensembledistil` (SPLADE++ EnsembleDistil), $w_j = \max_i \log(1 + \mathrm{ReLU}(o_{ij}))$. SciFact is encoded on CPU with Pyserini's `SpladeDocumentEncoder` (max length 256, integer-quantised weights) and indexed with `-impact -pretokenized`; the prebuilt SciFact index is run as a sanity check. Encoding 5M documents on CPU is not feasible, so FEVER and HotpotQA use Pyserini's prebuilt `beir-v1.0.0-*.splade-pp-ed` impact indexes (same checkpoint); queries are encoded locally with `SpladeQueryEncoder` and searched with `LuceneImpactSearcher`.

PART5DISCUSSION

Expansion terms: SPLADE vs. Rocchio vs. HyDE

For 12 sample queries per dataset: the top-10 vocabulary terms SPLADE weights beyond the literal query word-pieces, next to the top-10 Rocchio terms (4a, N=10, k=20) and the top-10

Dataset	(q,gold) pairs	Failure rate	Jaccard success (mean/med)	Jaccard failure (mean/med)	AUC

Table 6: Part 3: BM25 (tuned) failure = gold document not in the top-10. Jaccard overlap of analysed query and gold-document tokens; AUC = probability that a random success pair has higher overlap than a random failure pair.

Dataset	J=0	$0 < J < .05$	$.05 \leq J < .10$	$.10 \leq J < .20$	$J \geq .20$

Table 7: Part 3: failure rate (number of pairs) per Jaccard-overlap bin.

HyDE+Rocchio terms (4b, k=20). Overlap is counted after stemming all three lists with the index analyzer.

PART5OVERLAP

SPLADE (beyond query word-pieces)	Rocchio (4a, corpus PRF)	HyDE + Rocchio (4b)

Table 13: Part 5: top-10 expansion terms per source for the first 10 sample queries per dataset (Rocchio/HyDE terms are Porter-stemmed).

Use of LLM assistants

Claude Code (Anthropic, Claude Fable 5.1) was used as a coding assistant inside VS Code/WSL: it wrote the scripts from our instructions and the problem statement after inspecting the Pyserini API, ran them in our WSL environment, generated the LaTeX tables from the result files and drafted this report, which we checked against the numbers and edited. The local LLM used inside the experiments (Part 4b) is Qwen2.5-0.5B-Instruct, run offline. Shareable chat links: CHATLINKS.

Dataset	Model	nDCG@10	R@100	MRR@10	MAP

Table 8: Part 4a: own Rocchio (α=1, β=0.75, df-cutoff 10%) and Pyserini RM3 (original-query weight 0.5) at two (N, k) settings, vs. tuned BM25 on the same queries (FEVER / HotpotQA: fixed random sample of 500 test queries).

Dataset	Variant	nDCG@10	R@100	MRR@10	MAP

Table 9: Part 4b: LLM-generated feedback (Qwen2.5-0.5B-Instruct, 2 hypothetical documents per query) vs. corpus PRF (best 4a setting), same queries as Part 4a.

Dataset	Model	nDCG@10	R@100	MRR@10	MAP

Table 10: Part 5: SPLADE++ EnsembleDistil (`naver/splade-cocondenser-ensembledistil`) vs. tuned BM25 on the same queries.

Dataset	Impact index	Encoding time	Index time	Index size	Query latency (ms)

Table 11: Part 5: SPLADE impact indexes (latency = end-to-end per query incl. CPU query encoding, 8 search threads).

Dataset	queries	SPLADE∩Rocchio	SPLADE∩HyDE	Rocchio∩HyDE	all three

Table 12: Part 5: average number of shared expansion terms (top-10 per source, compared after Porter stemming).